Lab 1.1: Creating an MFC-based Application

In this lab, you will create, build, and run a simple MFC-based application in the Developer Studio® development environment using AppWizard.

To see a demonstration of the lab solution, click this icon.

(CD-ROM plays the demonstration, "Lab 1.1 Demonstration.")

Estimated time to complete this lab: **20 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The solution code for this lab is located in the folder \Labs\Ch01\Lab1.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Use AppWizard to create a simple Scribble application in MFC.
- Use Developer Studio and Visual C++® to build and run the application.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating a Simple Scribble Application

In this exercise, you will use Developer Studio, MFC AppWizard, and Visual C++ to create, build, and run a simple MFC single document interface (SDI) Scribble application.

27 Exercise 1: Creating a Simple Scribble Application

In this exercise, you will use Developer Studio, MFC AppWizard, and Visual C++ to create, build, and run a simple MFC-based single document interface (SDI) Scribble application.

▶ Use MFC AppWizard to create a new project

☐ 1. In Developer Studio, on the **File** menu, click **New**.

☐ 2. In the **New** dialog box, click the **Projects** tab, and then do the following:

a. For the project type, select **MFC AppWizard (exe)**.

b. For the project name, type **Scribble**.

c. Set the location for your project.

d. Accept the default platform **Win32**®.

e. Click **OK** to create the new project workspace.

MFC AppWizard starts.

☐ 3. In MFC AppWizard Step 1, click **Single document** and **English**, leave the **Document/View architecture support** checkbox selected, and then click **Next** to go to Step 2.

☐ 4. Because this project does not need database support, accept the default **None** for database support and click **Next** to go to Step 3.

☐ 5. In Step 3, accept **None** for the document support, clear the **ActiveX® Controls** check box, and then click **Next** to go to Step 4.

☐ 6. In Step 4, clear the **Printing and print preview** check box, accept the defaults (**Docking toolbar**, **Initial status bar**, **3D controls**, and 4 files for the recent file list) and then click **Next** to go to Step 5.

☐ 7. In Step 5, leave the project style as **MFC Standard**, click **Yes, please** for generating source file comments and select **As a shared DLL** for MFC support. Click **Next** to go to Step 6.

☐ 8. In Step 6, click **Finish** to display the **New Project Information** dialog box summarizing your choices.

☐ 9. To create the application files with MFC AppWizard, click **OK**.

When MFC AppWizard is finished, you will be returned to Developer Studio. To see the classes that MFC AppWizard created, click the **ClassView** tab in the Project Workspace window.

▶ Add message handlers for mouse messages

☐ 1. On the **View** menu, click **ClassWizard**.

The **MFC ClassWizard** property page appears.

☐ 2. In the **Message Maps** tab, in the combo boxes provided on the property page, select the **Scribble** project, the **CScribbleView** class, and the **CScribbleView** Object ID.

☐ 3. In the **Messages** list box, select the WM_LBUTTONDOWN message and click **Add Function**.

☐ 4. Repeat Step 3 for the WM_MOUSEMOVE and WM_LBUTTONUP messages.

☐ 5. Click **OK** to close ClassWizard.

Notes

▶ **Implement OnLButtonDown, OnMouseMove, and OnLButtonUp handlers**

☐ 1. Open ScribbleView.h.

☐ 2. In the public attributes section of the CScribbleView class declaration, define startpt and endpt as follows:

```
CPoint startpt, endpt;
```

☐ 3. Save ScribbleView.h.

☐ 4. Open ScribbleView.cpp.

☐ 5. In the constructor, initialize startpt and endpt coordinates to −1 as follows:

```
startpt=-1;
endpt=-1;
```

☐ 6. Scroll to the **OnLButtonDown** handler.

☐ 7. In the **OnLButtonDown** handler, save the point where the mouse button is pressed as the start point:

```
startpt.x=point.x;
startpt.y=point.y;
```

☐ 8. Scroll to the **OnMouseMove** handler.

☐ 9. In the **OnMouseMove** handler, add the following code to draw a line from the previous detected point in the mouse drag to the current point:

```
CClientDC dc(this);
endpt.x = point.x;
endpt.y = point.y;
if (startpt.x != -1 )
{
    dc.MoveTo(startpt.x, startpt.y);
    dc.LineTo(endpt.x, endpt.y);
    startpt.x = endpt.x;
    startpt.y = endpt.y;
}
```

☐ 10. In the **OnLButtonUp** handler, add code to reinitialize the variable startpt as follows:

```
startpt = -1;
```

▶ **Build and run the project**

☐ 1. On the **Build** menu, click **Build Scribble.exe**.

Developer Studio displays the status of the build process as it builds your project.

☐ 2. After the build is complete, on the **Build** menu, click **Execute Scribble.exe**.

The Scribble application starts.

The solution code for this exercise is located in the folder \Labs\Ch01\Lab1.1\Ex01\Solution.

Notes